

FSPM Assignment 3

Members:

Ayyan Ali (22k-5194)

Amal Abdul (22k-4822)

Ali Nazir (22k-5164)

1. Application of Project Management Knowledge Areas

For each subheading:

- State the **goal** of that knowledge area for your project.
- Mention **which tool(s)** you're using.
- Write 3–5 lines on **how you'll actually use them** in your project.

1.1. Project Integration Management

Goal in This Project

The goal of Project Integration Management for **Sahaara** is to ensure that all modules—help requests, alerts, help offers, user verification, map view, notifications, moderation, and communication—work together smoothly as one unified system. It coordinates planning, development, monitoring, and handling changes so the final product remains consistent and aligned with the project objectives.

Tools Used

- **Project Charter**
- **Project Management Plan**
- **Change Request Form**
- **Lessons Learned Register**

How It Will Be Applied

We will begin by creating a **Project Charter** outlining the purpose of Sahaara, its goals, major features, stakeholders, and high-level constraints. Next, a **Project Management Plan** will be drafted, containing timelines, development phases, task assignments, and risk strategies to keep the work structured. Any modifications to features—such as new verification rules, helper-matching logic, or UI changes—will be submitted through a **Change Request Form** and approved by the supervisor before implementation. Throughout the project, key insights or challenges will be logged in a **Lessons Learned Register**, ensuring continuous improvement and preventing repeated mistakes across development phases.

Here is the link to our Project Charter: [Project Charter](#)

1.2. Project Scope Management

The goal of Scope Management in **Sahaara** is to clearly define what features will be developed, what is excluded, and how the work will be broken down into manageable modules. This ensures the team stays focused on delivering the agreed-upon core functionality without unnecessary scope expansion.

Tools Used

- **Scope Statement**
- **Work Breakdown Structure (WBS)**
- **Scope Baseline**
- **Requirements Documentation**
- **Requirements Traceability Matrix (RTM)**

How It Will Be Applied

A detailed **Scope Statement** will be created, listing all essential features of Sahaara—help requests, alerts, help offers, nearby feed, live map, user verification, real-time chat, ratings, moderation, and smart notifications. Non-essential or future features such as AI-powered chatbots or multi-language support will be marked **out of scope** for this phase.

The project will be broken into modules using a **Work Breakdown Structure (WBS)** (e.g., frontend screens, backend APIs, database design, authentication, verification module, helper-matching module, etc.). A **Scope Baseline** combining the WBS, scope statement, and approved requirements will guide development.

All requirements will be documented and mapped in a **Requirements Traceability Matrix (RTM)** to ensure each feature is properly implemented and tested, preventing missing or misinterpreted requirements.

Here is our WBS:

i	Outlin Numb	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Resource Names
	1	★	Requirements & Documentation	400 days	Wed 20-08-25	Tue 02-03-27		
	1.1	★	Market Gap Analysis	9 days	Wed 20-08-25	Mon 01-09-25		Amal Abdul,Ayan Ali
	1.2	🔧	Market Survey	7 days	Tue 02-09-25	Wed 10-09-25	2	Ali Nazir
	1.3	🔧	Requirement Planning	8 days	Thu 11-09-25	Fri 26-09-25	2,3	Amal Abdul,Ayan Ali
	1.4	🔧	Documentation	7 days	Tue 28-04-26	Mon 07-09-26	4	Amal Abdul
	2	★	Design	254 days	Thu 25-09-25	Tue 15-09-26		
	2.1	🔧	Database Schema Design	3 days	Mon 29-09-25	Wed 01-10-25	4	Ayan Ali
	2.2	🔧	Wireframe + UI/UX	23 days	Thu 02-10-25	Tue 15-09-26	7	Amal Abdul
	3	★	Backend Development	655 days	Mon 29-09-25	Fri 31-03-28		
	3.1	🔧	BE integration with DB	2 days	Wed 29-03-28	Thu 30-03-28	7	Ayan Ali
	3.2	🔧	BE API Development	109 days	Thu 02-10-25	Fri 31-03-28	7	Ayan Ali
	4	★	Frontend Development	764 days	Mon 29-09-25	Thu 31-08-28		
	4.1	★	Mobile App Development	218 days	Tue 22-12-26	Thu 31-08-28	8	Ali Nazir,Ayan Ali
	4.2	🔧	Admin Panel Development	88 days	Wed 16-09-26	Fri 15-01-27	8	Amal Abdul
	5	★	Integration	60 days	Thu 01-01-26	Wed 25-03-26		
	5.1	★	App Integration with Backend	194 days	Fri 27-03-26	Wed 23-12-26	11	Ali Nazir,Ayan Ali
	5.2	★	Admin Panel Integration with Backend	86 days	Wed 25-02-26	Wed 30-09-26	11	Amal Abdul,Ayan Ali
	6	★	Testing	676 days	Wed 01-10-25	Wed 03-05-28		
	6.1	★	API Testing	109 days	Thu 24-12-26	Tue 25-05-27	11	Ayan Ali
	6.2	★	App Testing	218 days	Sun 26-10-25	Mon 25-10-27	13	Ali Nazir,Ayan Ali
	6.3	★	Admin Panel Testing	474 days	Mon 04-05-26	Thu 24-02-28	14	Amal Abdul,Ayan Ali
	6.4	★	End-to-End Testing	24 days	Fri 19-06-26	Wed 22-07-26	16,17	Ayan Ali
	6.5	★	User Acceptance Testing	23 days	Thu 25-02-27	Mon 29-03-27	22	Amal Abdul
	7	★	Deployment	23 days	Fri 25-02-28	Tue 28-03-28	22,23	Ayan Ali

1.3. Project Schedule (Time) Management

The goal of Schedule Management for **Sahaara** is to plan, sequence, and estimate all development activities so that the project is completed within the allotted semester timeline. It ensures the work is organized, dependencies are respected, and delays are identified early.

Tools Used

- **Activity List / Duration Estimates**
- **Network Diagram (PERT / Activity-on-Node)**
- **Critical Path Method (CPM)**
- **Gantt Chart**
- **Milestone List**

How It Will Be Applied

We will start by creating a detailed **Activity List** that covers tasks such as requirement gathering, UI design, backend API development, database creation, map integration, user verification module, testing, and deployment. Each task will be given a **duration estimate**, after which a **Network Diagram** will be constructed to map task dependencies. Using the **Critical Path Method (CPM)**, we will identify the longest path of dependent tasks that determine the minimum project completion time.

This information will then be converted into a **Gantt Chart** showing timelines, overlaps, dependencies, and major milestones such as “Frontend Prototype Complete,” “Core API Development Complete,” “Verification Module Complete,” and “Final Testing.” Regularly updating the Gantt chart will help track progress and ensure the project stays on schedule.

1.4. Project Cost Management

The goal of Cost Management for **Sahaara** is to estimate, budget, and control all project-related expenses to ensure the solution can be realistically developed within available resources. It helps establish a clear cost baseline against which the project's financial performance can be measured.

Tools Used

- **Cost Breakdown Structure (CBS)**
- **Cost Estimates**
- **Cost Baseline**
- **Expert Judgment**
- **Analogous Estimation**

How It Will Be Applied

A **Cost Breakdown Structure (CBS)** is developed to categorize and estimate all major project expenses, including labor, software tools, hardware, and miscellaneous overheads. Labor cost was estimated using analogous estimation based on three junior developers working over the project duration. Software/tooling costs (e.g., Google Maps API usage) and hardware costs (laptops and test devices) were added using market-based estimates.

These calculations allow us to prepare a **Cost Baseline**, which will serve as the official approved budget for the project. Throughout the development process, actual expenses will be compared against this baseline to ensure spending remains within limits and any variances are documented and addressed.

Here is the cost baseline for our project

Cost Category	Description / Assumptions	Cost (PKR)
Labor	3 junior developers × 13 person-months × 80,000 PKR/month	1,040,000
Software / Tools	Google Maps API	50,000
Hardware / Devices	3 laptops + testing devices	300,000
Miscellaneous	Internet, cloud hosting, admin costs, contingencies	60,000
Total Project Cost		1,450,000

1.5. Project Quality Management

The goal of Quality Management for **Sahaara** is to ensure that the system meets expected standards of functionality, reliability, usability, security, and performance. Quality processes will verify that all modules—such as help requests, alerts, chat, verification, and map view—work correctly and provide a smooth user experience.

Tools Used

- **Quality Management Plan**
- **Quality Checklists**
- **Test Plan / Test Cases**
- **Code Review Checklist**
- **Defect Log / Bug Tracker**

How It Will Be Applied

A **Quality Management Plan** will define the quality objectives, coding standards, and testing approach for the entire Sahaara project. Testing will be performed through structured **test plans**, including unit tests for API endpoints, integration tests for features like accepting

requests, and user testing for UI/UX flow.

Before code is merged, a **code review checklist** will be followed to ensure readability, maintainability, and security of the codebase. Any issues found during testing or reviews will be logged in a **Defect Log**, which will track bug status and ensure timely resolution.

A custom **Quality Checklist** will be used at each development milestone to ensure major features meet defined quality criteria before moving to the next phase.

Here the Quality checklist prepared for our project:

Quality Area	Checklist Item	Status
Functionality	All core features (help requests, offers, alerts, chat, ratings) working as per requirements	☐
	User verification (CNIC + selfie) implemented and functioning correctly	☐
	Smart notifications trigger correctly for relevant users	☐
Usability	UI screens are consistent, readable, and mobile-optimized	☐
	Navigation between feeds, map, and requests is smooth	☐
	Forms have proper validations and error messages	☐
Reliability and Performance	API response times under acceptable limits	☐
	App handles multiple concurrent users without failure	☐

	Requests and chat messages sync in real time	☐
Security	Authentication + authorization implemented correctly	☐
	Sensitive data (CNIC images, location) securely stored/transferred	☐
Testing and Review	Unit tests passed for major modules	☐
	Code reviewed using checklist before merging	☐
	All defects logged and resolved before release	☐

1.6. Project Resource Management

The goal of Resource Management for **Sahaara** is to plan, acquire, and manage all human and physical resources required for successful project completion. It ensures that team members have clear responsibilities and that essential tools, hardware, and software are available throughout development.

Tools Used

- **Resource Breakdown Structure (RBS)**
- **RACI Matrix**
- **Team Charter**
- **Resource Allocation Chart**

How It Will Be Applied

A **Resource Breakdown Structure (RBS)** will categorize all resources needed, including human roles (developers, UI designer, tester) and physical resources (laptops, development tools, APIs). A **Team Charter** will define team expectations, communication rules, and working guidelines.

A **RACI Matrix** will be created to assign responsibility and accountability for major project tasks such as UI design, API development, verification module, testing, and documentation.

This ensures everyone knows their role and avoids confusion.

Throughout the project, a resource allocation chart will help track the workload of each team member, ensuring balanced distribution and preventing bottlenecks.

Here is the RACI Matrix for our Project:

Project Task	Amal	Ayan	Ali	Supervisor
Requirements Gathering	R	C	C	A
UI/UX Design	A	R	C	I
Backend API Development	R	R	C	I
Database Design	A	R	C	I
User Verification Module	R	A	C	I
Smart Notifications Module	R	C	A	I
Frontend Mobile Screens	C	R	A	I
Testing (Unit + Integration)	R	A	C	I
Bug Fixing	R	R	C	I
Documentation & Report Writing	A	R	C	I

Final Deployment	R	A	C	I
-------------------------	---	---	---	---

- **R (Responsible):** Performs the task
- **A (Accountable):** Final decision-maker; owns the task
- **C (Consulted):** Provides input/expertise
- **I (Informed):** Needs updates but not directly involved

1.7. Project Communications Management

The goal of Communications Management for **Sahaara** is to ensure that all stakeholders receive the right information at the right time using the correct communication channels. This helps maintain alignment, track progress, resolve issues quickly, and keep the project running smoothly.

Tools Used

- **Communication Management Plan**
- **Stakeholder Communication Matrix**
- **Meeting Agenda & Minutes**
- **Status Report Template**

How It Will Be Applied

A **Communication Management Plan** will define how updates, risks, decisions, and documentation are shared among team members and the supervisor. Weekly or bi-weekly meetings will be conducted to discuss progress, blockers, and upcoming tasks, with minutes recorded for reference.

A **Stakeholder Communication Matrix** will be used to specify which stakeholder receives what type of information, how often, and through which medium (e.g., WhatsApp, email, in-person meetings). Status reports will be shared periodically to ensure transparency and consistent tracking of project milestones.

Here is the Stakeholder Communication Matrix for our project:

Stakeholder	Information Needed	Frequency	Communication Channel
--------------------	---------------------------	------------------	------------------------------

Supervisor	Project progress, issues, updated documents, milestone completion	Weekly	In-person meeting + Email
Development Team	Daily tasks, updates, blockers, technical decisions	Daily	WhatsApp Group + Stand-up Meeting
Tester / QA Role	Build updates, bug reports, testing schedules	Twice a week	WhatsApp + Google Drive
End Users (for testing phase)	Test instructions, feedback forms, updates on test builds	As needed	Email / WhatsApp
Admin Verification Stakeholder	Updates on verification module, security workflows	As required	WhatsApp + In-person meeting

1.8. Project Risk Management

The goal of Risk Management for **Sahaara** is to identify potential threats early, assess their likelihood and consequences, and prepare effective strategies to reduce their impact on project quality, schedule, and functionality. It ensures the project remains stable even if unexpected issues arise.

Tools Used

- **Risk Register**
- **Probability–Impact Matrix**
- **Risk Breakdown Structure (RBS)**
- **Risk Response Plan**

How It Will Be Applied

A **Risk Register** will be maintained throughout the project to document each risk, its cause, probability, impact, priority level, mitigation strategy, and assigned owner. Risks will be assessed using a **Probability–Impact Matrix** to classify them as low, medium, or high priority based on severity.

A **Risk Breakdown Structure (RBS)** will categorize risks (technical, schedule, resource, security, external). For each high-priority risk, a **Risk Response Plan**—including avoidance, mitigation, transfer, or acceptance—will be created. Risks will be reviewed weekly during team meetings to ensure timely action.

Here is the Probability-Impact Matrix for our project:

Impact ↓ / Probability →	Low	Medium	High
High Impact	Medium	High	Critical
Medium Impact	Low	Medium	High
Low Impact	Low	Low	Medium

Interpretation:

- **Critical:** Immediate action required; top priority risks
- **High:** Mitigation required; monitor frequently
- **Medium:** Mitigate if possible; track regularly
- **Low:** Accept or monitor occasionally

1.9. Project Procurement Management

The goal of Procurement Management for **Sahaara** is to identify which software, tools, or services need to be acquired from external sources and ensure they are selected based on cost, reliability, and suitability. This ensures that project development is efficient and avoids unnecessary expenses.

Tools Used

- **Make-or-Buy Analysis**
- **Procurement Management Plan**
- **Vendor Selection Criteria / Weighted Scoring Model**
- **Purchase Order (conceptual)**

How It Will Be Applied

A **Make-or-Buy Analysis** will be used to decide whether specific tools, APIs, or services should be developed in-house or procured externally. For instance, Google Maps integration and cloud hosting services will be procured externally due to complexity and reliability requirements, while modules like user verification, chat, and notifications will be developed in-house.

A **Procurement Management Plan** will document all external acquisitions, estimated costs, approval process, and timelines. **Vendor Selection Criteria** will prioritize cost-effectiveness, technical reliability, and support services for any paid APIs or services. Conceptual **Purchase Orders** will formalize the acquisition decisions.

Here is a Make-or-Buy Analysis for our project:

Component / Tool	Make (In-house)	Buy (External Service)	Decision	Reason
User Verification Module	✓		Make	Requires custom integration with CNIC + selfie validation; high security control needed
Real-time Chat & Messaging	✓		Make	Custom implementation needed to integrate with multiple responders and verification system
Smart Notification Engine	✓		Make	Needs custom prioritization based on proximity, skills, and past history
Google Maps Integration		✓	Buy	Reliable mapping and geolocation features; external API is cost-effective and scalable
Cloud Hosting (Server / Database)		✓	Buy	Outsourcing ensures uptime, scalability, and minimal maintenance overhead
Image Storage / CDN		✓	Buy	Efficient handling of verified user images and alerts requires secure cloud storage

1.10. Project Stakeholder Management

The goal of Stakeholder Management for **Sahaara** is to identify all individuals or groups affected by or interested in the project, understand their influence and interest, and plan communication and engagement strategies to ensure smooth development and adoption of the platform.

Tools Used

- **Stakeholder Register**
- **Stakeholder Analysis Grid (Power–Interest Matrix)**
- **Stakeholder Engagement Matrix**

How It Will Be Applied

A **Stakeholder Register** will list all key stakeholders, their roles, interests, and influence. Each stakeholder will be analyzed using a **Power–Interest Matrix** to categorize them based on their ability to affect the project and their level of interest.

Based on this categorization, engagement strategies will be defined (e.g., keep satisfied, keep informed, manage closely). Regular communication and updates will be planned according to these strategies to ensure stakeholder expectations are met, risks of misalignment are minimized, and feedback is incorporated effectively.

Here is the Power-Interest Matrix for our project:

Power / Interest →	Low Interest	High Interest
High Power	Keep Satisfied	Manage Closely
Low Power	Monitor	Keep Informed

Stakeholder Placement for our project:

Stakeholder	Power	Interest	Matrix Quadrant / Engagement Strategy

Supervisor / FYP Advisor	High	High	Manage Closely – frequent updates and approvals
Development Team	Medium	High	Keep Informed – regular stand-ups, task assignments
Testers / QA	Medium	Medium	Keep Informed – provide build updates and testing schedules
End Users / Volunteers	Low	High	Keep Informed – gather feedback through surveys and beta testing
Admin	High	Medium	Keep Satisfied – provide important updates on user verification, reports, and moderation tools
Vendors / API Providers	Medium	Low	Monitor – maintain technical communication for service reliability

2. Project Crashing and Cost Calculation

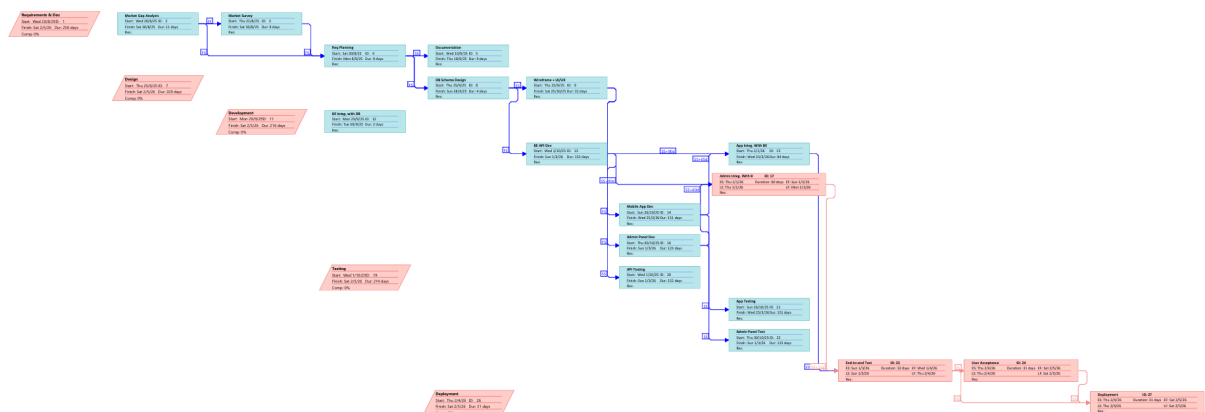
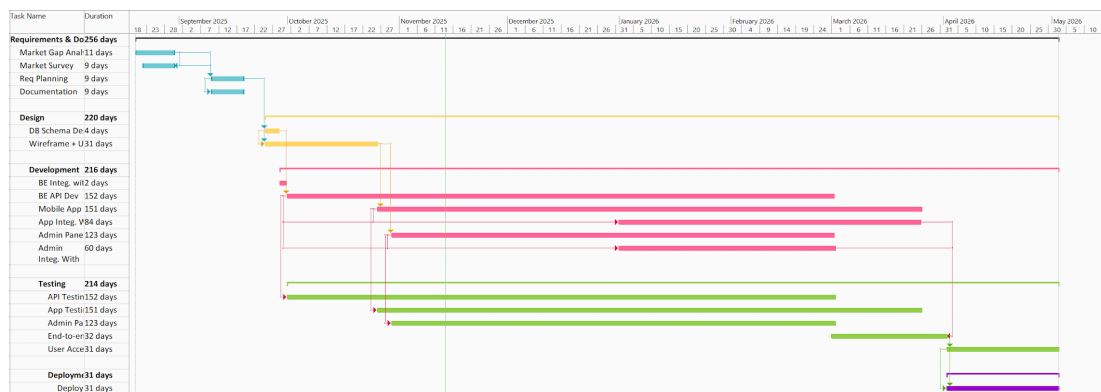
What to do overall:

- Build a small schedule model (activities, durations, costs).
- Identify the critical path
- Crash activities step by step, recalculate duration and cost, and summarize the result.

2.1. Activity List and Network (Normal Schedule)

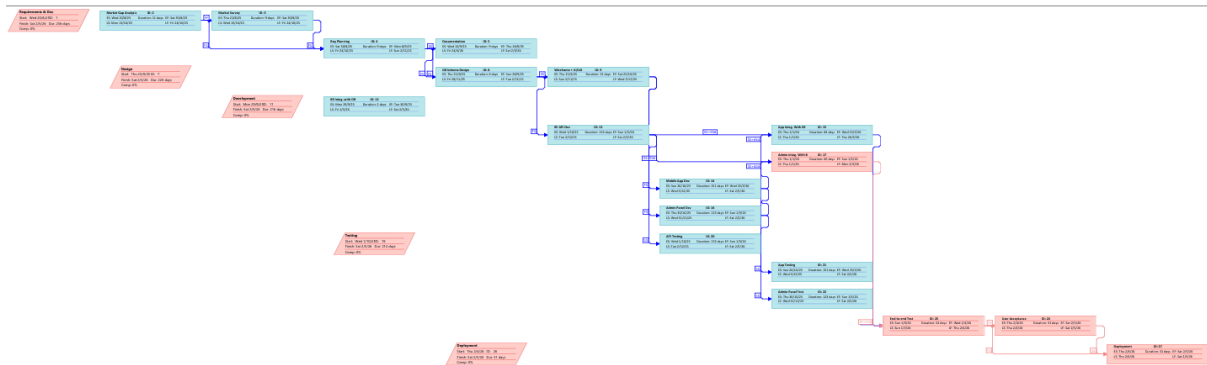
A table with activities, normal duration, dependencies, and a simple network diagram

ID	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Phase
1		Requirements & Doc	256 days	Wed 20/8/25	Sat 2/5/26		
2		Market Gap Analysis	11 days	Wed 20/8/25	Sat 30/8/25		Requirements & Doc
3		Market Survey	9 days	Thu 21/8/25	Sat 30/8/25	2FF	Requirements & Doc
4		Req Planning	9 days	Sat 30/8/25	Mon 8/9/25	2,3	Requirements & Doc
5		Documentation	9 days	Wed 10/9/25	Thu 18/9/25	4SS	Requirements & Doc
6							
7		Design	220 days	Thu 25/9/25	Sat 2/5/26		
8		DB Schema Design	4 days	Thu 25/9/25	Sun 28/9/25	4	Design
9		Wireframe + UI/UX	31 days	Thu 25/9/25	Sat 25/10/25	4,8SS	Design
10							
11		Development	216 days	Mon 29/9/25	Sat 2/5/26		
12		BE Integ. with DB	2 days	Mon 29/9/25	Tue 30/9/25		Development
13		BE API Dev	152 days	Wed 1/10/25	Sun 1/3/26	8	Development
14		Mobile App Dev	151 days	Sun 26/10/25	Wed 25/3/26	9	Development
15		App Integ. With BE	84 days	Thu 1/1/26	Wed 25/3/26	13SS+90 days,14SS+	Development
16		Admin Panel Dev	123 days	Thu 30/10/25	Sun 1/3/26	9	Development
17		Admin Integ. With BE	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 days,13SS+	Development
18							
19		Testing	214 days	Wed 1/10/25	Sat 2/5/26		
20		API Testing	152 days	Wed 1/10/25	Sun 1/3/26	13SS	Testing
21		App Testing	151 days	Sun 26/10/25	Wed 25/3/26	14SS	Testing
22		Admin Panel Test	123 days	Thu 30/10/25	Sun 1/3/26	16SS	Testing
23		End-to-end Test	32 days	Sun 1/3/26	Wed 1/4/26	17FF+31 days,15FF+	Testing
24		User Acceptance	31 days	Thu 2/4/26	Sat 2/5/26	23	Testing
25							
26		Deployment	31 days	Thu 2/4/26	Sat 2/5/26		
27		Deployment	31 days	Thu 2/4/26	Sat 2/5/26	23,24SS	Deployment

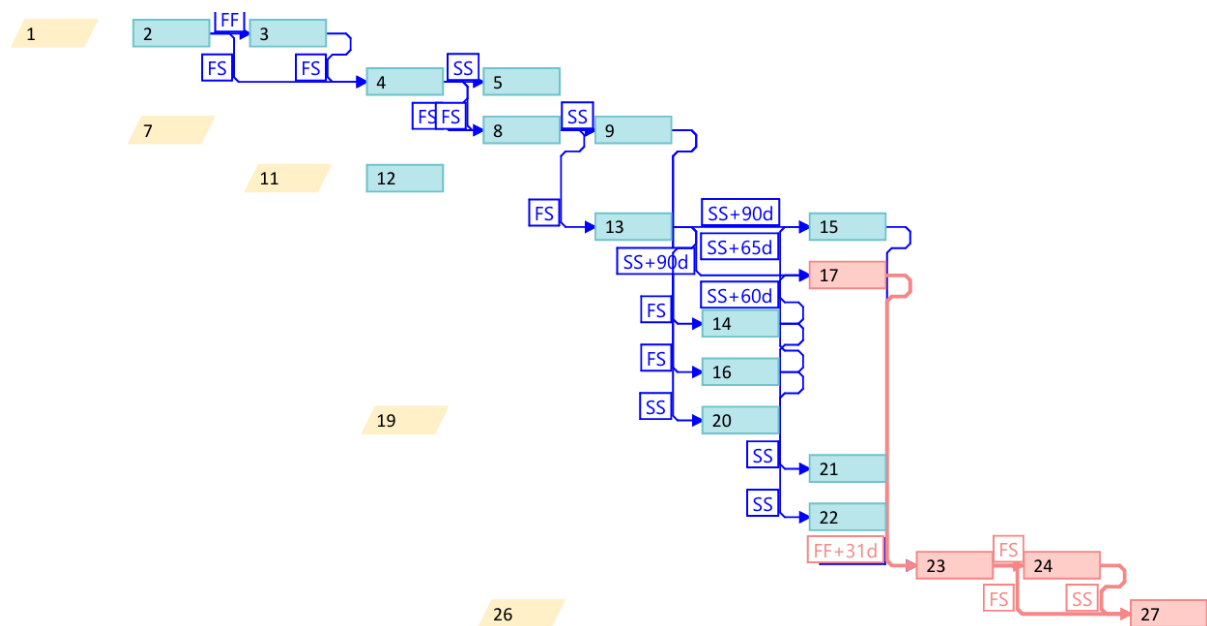


2.2. Critical Path Identification (Normal Plan)

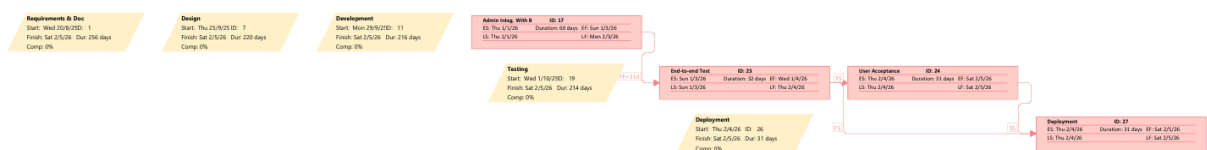
We computed early start/finish, late start/finish, and identified the critical path.



AON:



Critical Path:



2.3. Crashing Data Setup

For each activity, defined normal duration & cost, crashed duration & cost to calculate the crash slope

Crash Duration

The shortest possible time a task can be completed after adding extra resources. It is less than the normal duration.

Crash Cost

The total cost of a task when it is performed at crash duration. Includes overtime, extra labor, faster tools, etc.

Extra Cost (Crash Cost – Normal Cost)

The additional amount of money required to crash the activity.

Crash Cost Slope

The rate of additional cost per day saved when a task is crashed.

$$\text{Crash Cost Slope} = (\text{Normal Duration} - \text{Crash Duration}) / (\text{Crash Cost} - \text{Normal Cost})$$

ID	Task Name	Duration	Labor Cost	Software Cost	Hardware Cost	Misc Cost	Total Cost	Crash Duration	Days Saved	Extra Cost	Crash Cost
1	Requirements & Doc	256 days	Rs0	Rs0	Rs0	Rs0	Rs0	0 days	0 days	Rs0	Rs0
2	Market Gap Analysis	11 days	Rs9,820	Rs724	Rs0	Rs567	Rs11,111	9 days	2 days	Rs2,678	Rs13,788
3	Market Survey	10 days	Rs8,034	Rs592	Rs0	Rs464	Rs9,090	8 days	2 days	Rs2,678	Rs11,768
4	Req Planning	9 days	Rs8,034	Rs592	Rs0	Rs464	Rs9,090	7 days	2 days	Rs2,678	Rs11,768
5	Documentation	9 days	Rs8,034	Rs592	Rs0	Rs464	Rs9,090	7 days	2 days	Rs2,678	Rs11,768
6											
7	Design	220 days	Rs0	Rs0	Rs0	Rs0	Rs0	0 days	0 days	Rs0	Rs0
8	DB Schema Design	4 days	Rs3,571	Rs857	Rs0	Rs206	Rs4,634	3 days	1 day	Rs1,339	Rs5,973
9	Wireframe + UI/UX	31 days	Rs27,674	Rs6,642	Rs0	Rs1,597	Rs35,913	25 days	6 days	Rs4,159	Rs40,072
10											
11	Development	216 days	Rs0	Rs0	Rs0	Rs0	Rs0	0 days	0 days	Rs0	Rs0
12	BE Integ. with DB	2 days	Rs1,785	Rs105	Rs734	Rs104	Rs2,728	1 day	1 day	Rs1,339	Rs4,067
13	BE API Dev	152 days	Rs135,691	Rs7,972	Rs55,805	Rs7,828	Rs207,296	122 days	30 days	Rs71,405	Rs278,701
14	Mobile App Dev	151 days	Rs134,798	Rs7,920	Rs55,437	Rs7,777	Rs205,932	121 days	30 days	Rs67,981	Rs273,913
15	App Integ. With BE	84 days	Rs74,987	Rs4,406	Rs30,839	Rs4,326	Rs114,558	67 days	17 days	Rs25,471	Rs140,029
16	Admin Panel Dev	123 days	Rs109,803	Rs6,451	Rs45,157	Rs6,335	Rs167,746	98 days	25 days	Rs34,451	Rs202,197
17	Admin Integ. With BE	60 days	Rs53,562	Rs3,147	Rs22,028	Rs3,090	Rs81,827	48 days	12 days	Rs14,211	Rs96,038
18											
19	Testing	214 days	Rs0	Rs0	Rs0	Rs0	Rs0	0 days	0 days	Rs0	Rs0
20	API Testing	152 days	Rs135,691	Rs2,331	Rs23,313	Rs7,828	Rs169,163	122 days	30 days	Rs17,372	Rs186,535
21	App Testing	151 days	Rs134,798	Rs2,316	Rs23,160	Rs7,777	Rs168,051	121 days	30 days	Rs16,944	Rs184,995
22	Admin Panel Test	123 days	Rs109,803	Rs1,886	Rs18,865	Rs6,335	Rs136,889	98 days	25 days	Rs27,377	Rs164,266
23	End-to-end Test	32 days	Rs28,567	Rs490	Rs4,908	Rs1,648	Rs35,613	26 days	6 days	Rs3,405	Rs39,018
24	User Acceptance	31 days	Rs27,674	Rs475	Rs4,755	Rs1,596	Rs34,500	25 days	6 days	Rs3,312	Rs37,812
25											
26	Deployment	31 days	Rs0	Rs0	Rs0	Rs0	Rs0	0 days	0 days	Rs0	Rs0
27	Deployment	31 days	Rs27,674	Rs2,500	Rs15,000	Rs1,596	Rs46,770	25 days	6 days	Rs7,049	Rs53,819
28											
29	Total		Rs1,040,000	Rs50,000	Rs300,000	Rs60,000	Rs1,450,000	0 days	233 days	Rs312,000	Rs1,762,000

ID	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Cost	Crash Duration	Crash Cost	Crash Slope
1		Requirements & Doc	256 days	Wed 20/8/25	Sat 2/5/26		Rs2,900,001	0 days	Rs0	0
2		Market Gap Analysis	11 days	Wed 20/8/25	Sat 30/8/25		Rs11,111	9 days	Rs13,788	2.79
3		Market Survey	10 days	Thu 21/8/25	Sun 31/8/25	2FF	Rs9,090	8 days	Rs11,768	2.79
4		Req Planning	9 days	Sat 30/8/25	Mon 8/9/25	2,3	Rs9,090	7 days	Rs11,768	2.79
5		Documentation	9 days	Wed 10/9/25	Thu 18/9/25	4SS	Rs9,090	7 days	Rs11,768	2.79
6										
7		Design	220 days	Thu 25/9/25	Sat 2/5/26		Rs2,861,620	0 days	Rs0	0
8		DB Schema Design	4 days	Thu 25/9/25	Sun 28/9/25	4	Rs4,634	3 days	Rs5,973	2.79
9		Wireframe + UI/UX	31 days	Thu 25/9/25	Sat 25/10/25	4,8SS	Rs35,913	25 days	Rs40,072	1.44
10										
11		Development	216 days	Mon 29/9/25	Sat 2/5/26		Rs2,821,073	0 days	Rs0	0
12		BE Integ. with DB	2 days	Mon 29/9/25	Tue 30/9/25		Rs2,728	1 day	Rs4,067	2.79
13		BE API Dev	152 days	Wed 1/10/25	Sun 1/3/26	8	Rs207,296	122 days	Rs278,701	4.96
14		Mobile App Dev	151 days	Sun 26/10/25	Wed 25/3/26	9	Rs205,932	121 days	Rs273,913	4.72
15		App Integ. With BE	84 days	Thu 1/1/26	Wed 25/3/26	13SS+90 days	Rs114,558	67 days	Rs140,029	3.12
16		Admin Panel Dev	123 days	Thu 30/10/25	Sun 1/3/26	9	Rs167,746	98 days	Rs202,197	2.87
17		Admin Integ. With BE	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 days	Rs81,827	48 days	Rs96,038	2.47
18										
19		Testing	214 days	Wed 1/10/25	Sat 2/5/26		Rs2,040,986	0 days	Rs0	0
20		API Testing	152 days	Wed 1/10/25	Sun 1/3/26	13SS	Rs169,163	122 days	Rs186,535	1.21
21		App Testing	151 days	Sun 26/10/25	Wed 25/3/26	14SS	Rs168,051	121 days	Rs184,995	1.18
22		Admin Panel Test	123 days	Thu 30/10/25	Sun 1/3/26	16SS	Rs136,889	98 days	Rs164,266	2.28
23		End-to-end Test	32 days	Sun 1/3/26	Wed 1/4/26	17FF+31 days	Rs35,613	26 days	Rs39,018	1.18
24		User Acceptance	31 days	Thu 2/4/26	Sat 2/5/26	23	Rs34,500	25 days	Rs37,812	1.15
25										
26		Deployment	31 days	Thu 2/4/26	Sat 2/5/26		Rs1,496,770	0 days	Rs0	0
27		Deployment	31 days	Thu 2/4/26	Sat 2/5/26	23,24SS	Rs46,770	25 days	Rs53,819	2.45
28										
29		Total					Rs1,450,000	0 days	Rs1,762,000	#ERROR

2.4. Crashing Steps and New Critical Path(s)

Iteratively:

- Chooosed the cheapest activity on the current critical path.
- Crashed it by 1 day

- Recalculated the critical path and total duration.
- Record the extra cost on a separate table.

Critical Tasks:

ID	Task Mode	Task Name	Duration	Start	Finish	Predecessors	25 9	Dec '25 16	23	30	7	14	21	28	4	11	18	25	1	8	15	22	1	8	15	22	29	5	12	19	26	3	10
1		Requirements & Do	256 days	Wed 20/8/25	Sat 2/5/26																												
7		Design	220 days	Thu 25/9/25	Sat 2/5/26																												
11		Development	216 days	Mon 29/9/25	Sat 2/5/26																												
17		Admin Integ. With	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 days,13SS																											
19		Testing	214 days	Wed 1/10/25	Sat 2/5/26																												
23		End-to-en	32 days	Sun 1/3/26	Wed 1/4/26	17FF+31 d																											
24		User Acce	31 days	Thu 2/4/26	Sat 2/5/26	23																											
26		Deployment	31 days	Thu 2/4/26	Sat 2/5/26	23,24SS																											
27		Deploy	31 days	Thu 2/4/26	Sat 2/5/26	23,24SS																											

Project Duration Before crash iteration: 256 days

1st iteration:

ID	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Cost	Crash Duration	Crash Cost	Crash Slope	Days Crashed This Iter
1		Requirements & Doc	256 days	Wed 20/8/25	Sat 2/5/26		Rs2,900,001	0 days	Rs0	0	0
7		Design	220 days	Thu 25/9/25	Sat 2/5/26		Rs2,861,620	0 days	Rs0	0	0
11		Development	216 days	Mon 29/9/25	Sat 2/5/26		Rs2,821,073	0 days	Rs0	0	0
17		Admin Integ. With BE	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 days	Rs81,827	48 days	Rs96,038	2.47	0
19		Testing	214 days	Wed 1/10/25	Sat 2/5/26		Rs2,040,986	0 days	Rs0	0	0
23		End-to-end Test	32 days	Sun 1/3/26	Wed 1/4/26	17FF+31 days	Rs35,613	26 days	Rs39,018	1.18	0
24		User Acceptance	31 days	Thu 2/4/26	Sat 2/5/26	23	Rs34,500	25 days	Rs37,812	1.15	0
26		Deployment	31 days	Thu 2/4/26	Sat 2/5/26		Rs1,496,770	0 days	Rs0	0	0
27		Deployment	31 days	Thu 2/4/26	Sat 2/5/26	23,24SS	Rs46,770	25 days	Rs53,819	2.45	0

We will see the activity with the smallest crash slope to crash

We will confirm if the task can still be crashed

Check Duration and Crash Duration:

If Duration > Crash Duration → you can crash at least 1 day.

If Duration = Crash Duration → cannot crash this task; pick the next cheapest.

Yes, 31 > 25, so crashing can be done

	End-to-end Test	32 days	Sun 1/3/26	Wed 1/4/26	17FF+31 days	Rs35,613	26 days	Rs39,018	1.18	0
	User Acceptance	30 days	Thu 2/4/26	Fri 1/5/26	23	Rs34,500	25 days	Rs37,812	1.15	1

Calculated and recorded extra cost for this iteration, along with recomputation & inspection for the new critical path and project duration:

	A	B	C	D	E	F	G	H
1	Iteration	Activity Crashed	Time Reduced (days)	Crash Slope	Extra Cost This Iteration	New Project Duration	Cumulative Extra Cost	Chnged Critical Path
2	1	User Acceptance	1	1.15	1.15	256	1.15	No change
3								
4								

To decide whether to stop or continue

Stop if:

- You reached your target project duration, OR
- There are no more crashable tasks on any critical path, OR
- You have reached your crash budget.

Otherwise, go back to Step 1 and repeat.

2nd iteration:

	Task Mode	Task Name	Duration	Start	Finish	Predecessor	Cost	Crash Duration	Crash Cost	Crash Slope	Days Crashed This Iter
1		Requirements & Doc	256 days	Wed 20/8/25	Sat 2/5/26		Rs2,900,001	0 days	Rs0	0	0
7		Design	220 days	Thu 25/9/25	Sat 2/5/26		Rs2,861,620	0 days	Rs0	0	0
11		Development	216 days	Mon 29/9/25	Sat 2/5/26		Rs2,821,073	0 days	Rs0	0	0
17		Admin Integ. With I	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 day	Rs81,827	48 days	Rs96,038	2.47	0
19		Testing	214 days	Wed 1/10/25	Sat 2/5/26		Rs2,040,986	0 days	Rs0	0	0
23		End-to-end Test	32 days	Sun 1/3/26	Wed 1/4/26	17FF+31 day	Rs35,613	26 days	Rs39,018	1.18	0
24		User Acceptance	30 days	Thu 2/4/26	Fri 1/5/26	23	Rs34,500	25 days	Rs37,812	1.15	1
26		Deployment	31 days	Thu 2/4/26	Sat 2/5/26		Rs1,496,770	0 days	Rs0	0	0
27		Deployment	31 days	Thu 2/4/26	Sat 2/5/26	23,24SS	Rs46,770	25 days	Rs53,819	2.45	0

In the first crashing iteration, User Acceptance was the cheapest activity on the critical path, so I crashed it by 1 day. But, crashing it again in later iterations would not reduce the overall project duration, because the project finish date is controlled by the Deployment activity that comes after it. Deployment has a fixed finish date and cannot be shortened through User Acceptance. So even though User Acceptance still shows as a critical task, crashing it more provides no benefit to the total project duration.

For this reason, I ignored it in the next iterations and moved on to other critical tasks that actually influence the project's end date.

	Task Mode	Task Name	Duration	Start	Finish	Predecessor	Cost	Crash Duration	Crash Cost	Crash Slope	Days Crashed This Iter
1		Requirements & Doc	256 days	Wed 20/8/25	Sat 2/5/26		Rs2,900,001	0 days	Rs0	0	0
7		Design	220 days	Thu 25/9/25	Sat 2/5/26		Rs2,861,620	0 days	Rs0	0	0
11		Development	216 days	Mon 29/9/25	Sat 2/5/26		Rs2,821,073	0 days	Rs0	0	0
17		Admin Integ. With I	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 day	Rs81,827	48 days	Rs96,038	2.47	0
19		Testing	214 days	Wed 1/10/25	Sat 2/5/26		Rs2,040,986	0 days	Rs0	0	0
23		End-to-end Test	32 days	Sun 1/3/26	Wed 1/4/26	17FF+31 day	Rs35,613	26 days	Rs39,018	1.18	0
24		User Acceptance	30 days	Thu 2/4/26	Fri 1/5/26	23	Rs34,500	25 days	Rs37,812	1.15	1
26		Deployment	31 days	Thu 2/4/26	Sat 2/5/26		Rs1,496,770	0 days	Rs0	0	0
27		Deployment	31 days	Thu 2/4/26	Sat 2/5/26	23,24SS	Rs46,770	25 days	Rs53,819	2.45	0

End-to-end Test	30 days	Tue 3/3/26	Wed 1/4/26	17FF+31 day	Rs35,613	26 days	Rs39,018	1.18	0
-----------------	---------	------------	------------	-------------	----------	---------	----------	------	---

	Task Mode	Task Name	Duration	Start	Finish	Predecessor	Cost	Crash Duration	Crash Cost	Crash Slope	Days Crashed This Iter
1		Requirements & Doc	256 days	Wed 20/8/25	Sat 2/5/26		Rs2,900,001	0 days	Rs0	0	0
7		Design	220 days	Thu 25/9/25	Sat 2/5/26		Rs2,861,620	0 days	Rs0	0	0
11		Development	216 days	Mon 29/9/25	Sat 2/5/26		Rs2,821,073	0 days	Rs0	0	0
17		Admin Integ. With I	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 day	Rs81,827	48 days	Rs96,038	2.47	0
19		Testing	214 days	Wed 1/10/25	Sat 2/5/26		Rs2,040,986	0 days	Rs0	0	0
23		End-to-end Test	30 days	Tue 3/3/26	Wed 1/4/26	17FF+31 day	Rs35,613	26 days	Rs39,018	1.18	0
24		User Acceptance	30 days	Thu 2/4/26	Fri 1/5/26	23	Rs34,500	25 days	Rs37,812	1.15	0
26		Deployment	31 days	Thu 2/4/26	Sat 2/5/26		Rs1,496,770	0 days	Rs0	0	0
27		Deployment	31 days	Thu 2/4/26	Sat 2/5/26	23,24SS	Rs46,770	25 days	Rs53,819	2.45	0

	A	B	C	D	E	F	G	H
1	Iteration	Activity Crashed	Time Reduced (days)	Crash Slope	Extra Cost This Iteration	New Project Duration	Cumulative Extra Cost	Chnged Critical Path
2		1 User Acceptance	1	1.15	1.15	256	1.15	No change
3		2 End-To-End Testing	2	1.18	2.36	256	3.51	No change
4					0			

Again, no change in total duration - crashing is being wasted

Even though “User Acceptance” and “End-to-End Testing” appear critical, they do not drive the final project duration. Crashing them gives zero reduction, so I skip them and move to the next task with the lowest crash slope for the next iteration.

3rd Iteration:

	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Cost	Crash Duration	Crash Cost	Crash Slope	Days Crashed This Iter	Add New Column
1		Requirements & Doc	256 days	Wed 20/8/25	Sat 2/5/26		R\$2,900,001	0 days	R\$0	0	0	
7		Design	220 days	Thu 25/9/25	Sat 2/5/26		R\$2,861,620	0 days	R\$0	0	0	
11		Development	216 days	Mon 29/9/25	Sat 2/5/26		R\$2,821,073	0 days	R\$0	0	0	
15		App Integ. With BE	84 days	Thu 1/1/26	Wed 25/3/26	13SS+90 day	R\$114,558	67 days	R\$140,029	3.12	0	
17		Admin Integ. With I	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 day	R\$81,827	48 days	R\$96,038	2.47	0	
19		Testing	214 days	Wed 1/10/25	Sat 2/5/26		R\$2,040,986	0 days	R\$0	0	0	
23		End-to-end Test	30 days	Tue 3/3/26	Wed 1/4/26	17FF+31 day	R\$35,613	26 days	R\$39,018	1.18	0	
24		User Acceptance	30 days	Thu 2/4/26	Fri 1/5/26	23	R\$34,500	25 days	R\$37,812	1.15	0	
26		Deployment	31 days	Thu 2/4/26	Sat 2/5/26		R\$1,496,770	0 days	R\$0	0	0	
27		Deployment	31 days	Thu 2/4/26	Sat 2/5/26	23,24SS	R\$46,770	25 days	R\$53,819	2.45	0	

	Task Mode	Task Name	Duration	Start	Finish	Predecessors	Cost	Crash Duration	Crash Cost	Crash Slope	Days Crashed This Iter
1		Requirements & Doc	255 days	Wed 20/8/25	Fri 1/5/26		R\$2,900,001	0 days	R\$0	0	0
7		Design	219 days	Thu 25/9/25	Fri 1/5/26		R\$2,861,620	0 days	R\$0	0	0
11		Development	215 days	Mon 29/9/25	Fri 1/5/26		R\$2,821,073	0 days	R\$0	0	0
15		App Integ. With BE	84 days	Thu 1/1/26	Wed 25/3/26	13SS+90 day	R\$114,558	67 days	R\$140,029	3.12	0
17		Admin Integ. With I	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 day	R\$81,827	48 days	R\$96,038	2.47	0
19		Testing	213 days	Wed 1/10/25	Fri 1/5/26		R\$2,040,986	0 days	R\$0	0	0
23		End-to-end Test	30 days	Tue 3/3/26	Wed 1/4/26	17FF+31 day	R\$35,613	26 days	R\$39,018	1.18	0
24		User Acceptance	30 days	Thu 2/4/26	Fri 1/5/26	23	R\$34,500	25 days	R\$37,812	1.15	0
26		Deployment	30 days	Thu 2/4/26	Fri 1/5/26		R\$1,496,770	0 days	R\$0	0	0
27		Deployment	30 days	Thu 2/4/26	Fri 1/5/26	23,24SS	R\$46,770	25 days	R\$53,819	2.45	1

	A	B	C	D	E	F	G	H
1	Iteration	Activity Crashed	Time Reduced (days)	Crash Slope	Extra Cost This Iteration	New Project Duration	Cumulative Extra Cost	Chnged Critical Path
2		1 User Acceptance	1	1.15	1.15	256	1.15	No change
3		2 End-To-End Testing	2	1.18	2.36	256	3.51	No change
4		3 Deployment	1	2.45	2.45	255	5.96	1 day reduced

Because User Acceptance Testing (UAT) and Deployment are both ending tasks, each of them independently controls the project's final finish date. After the first crash, Deployment and UAT still finish on different days, so reducing Deployment's duration further does not shorten the overall project duration, as UAT now determines the project end. Crashing Deployment alone would therefore waste cost with no benefit. To actually reduce the total duration, both ending tasks would need to be crashed together, in alternating iterations, until one of them reaches its crash limit. Only then will the project's finish date move earlier.

Next two iterations:

1.

	End-to-end Test	30 days	Tue 3/3/26	Wed 1/4/26	17FF+31 day	R\$35,613	26 days	R\$39,018	1.18	0
	User Acceptance	26 days	Thu 2/4/26	Mon 27/4/26	23	R\$34,500	25 days	R\$37,812	1.15	0

2.

	Deployment	26 days	Thu 2/4/26	Mon 27/4/26	23,24SS	R\$46,770	25 days	R\$53,819	2.45	1
--	------------	---------	------------	-------------	---------	-----------	---------	-----------	------	---

	Requirements & Doc	251 days	Wed 20/8/25	Mon 27/4/26		R\$2,900,001	0 days	R\$0	0	0
	Design	215 days	Thu 25/9/25	Mon 27/4/26		R\$2,861,620	0 days	R\$0	0	0
	Development	211 days	Mon 29/9/25	Mon 27/4/26		R\$2,821,073	0 days	R\$0	0	0
	App Integ. With BE	84 days	Thu 1/1/26	Wed 25/3/26	13SS+90 day	R\$114,558	67 days	R\$140,029	3.12	0
	Admin Integ. With I	60 days	Thu 1/1/26	Sun 1/3/26	16SS+60 day	R\$81,827	48 days	R\$96,038	2.47	0
	Testing	209 days	Wed 1/10/25	Mon 27/4/26		R\$2,040,986	0 days	R\$0	0	0
	End-to-end Test	30 days	Tue 3/3/26	Wed 1/4/26	17FF+31 day	R\$35,613	26 days	R\$39,018	1.18	0
	User Acceptance	26 days	Thu 2/4/26	Mon 27/4/26	23	R\$34,500	25 days	R\$37,812	1.15	0
	Deployment	26 days	Thu 2/4/26	Mon 27/4/26		R\$1,496,770	0 days	R\$0	0	0
	Deployment	26 days	Thu 2/4/26	Mon 27/4/26	23,24SS	R\$46,770	25 days	R\$53,819	2.45	1

	A	B	C	D	E	F	G	H
1	Iteration	Activity Crashed	Time Reduced (days)	Crash Slope	Extra Cost This Iteration	New Project Duration	Cumulative Extra Cost	Chnged Critical Path
2		1 User Acceptance	1	1.15	1.15	256	1.15	No change
3		2 End-To-End Testing	2	1.18	2.36	256	3.51	No change
4		3 Deployment	1	2.45	2.45	255	5.96	1 day reduced
5		4 User Acceptance	4	1.15	4.6	255	10.56	No change
6		5 Deployment	4	2.45	9.8	251	20.36	4 Days reduced

Project Duration Before Crash Iteration: 256 days

Project Duration After Crash Iteration: 251 days

Cumulative Crashing Cost: 20.36

Final Cost = Original Total Cost + Cumulative Extra Crash Cost

Final Cost = Rs 1,450,000 + Rs 20.36 = Rs 1,450,020.36

3. Software Estimation

We are using Functional Point Analysis Estimation Method

3.1. Estimation Method and Rationale

Method: Function Point Analysis (FPA)

Rationale:

- Our project is a mobile-based app with clear functional modules.
- FPA works well at early stages, before code is written.
- Counting Inputs, Outputs, Inquiries, and Files is straightforward from our feature list.

3.2. Size Estimation

3.2.1. External Inputs (EI)

- Post a Help Request
- Post an Alert
- Post a Help Offer
- Accept Request (updates status)
- Verification uploads (CNIC + selfie)

Total EI: 5

3.2.2. External Outputs (EO)

- Nearby Requests/Offers/Alerts Feed
- Live Map View
- Rating/Trust Scores Display
- Notification System (Smart Notifications, Expiry alerts)

Total EO: 4

3.2.3. External Inquiries (EQ)

- View nearby requests/offers/alerts
- Filter Requests (by category, urgency, distance)
- Request Status Tracking (accepted, in progress, completed)

Total EQ: 3

3.2.4. Internal Logical Files (ILF)

- Users
- Requests
- Alerts
- Offers

- Ratings / Reviews
- Skills
- Verification Data
- History Logs
- Temporary Groups (multi-responder handling)

Total ILF: 9

3.2.5. External Interface Files (EIF)

- External API for geolocation (maps)
- Optional external verification API (if any)
- Possibly notification service (Push/Email)

Total EIF: 3

3.2.6. Unadjusted Complexity Weights

Component	Count	Complexity	Weight	Calculation
EI	5	Average	4	$5 \times 4 = 20$
EO	4	Average	5	$4 \times 5 = 20$
EQ	3	Average	4	$3 \times 4 = 12$
ILF	9	Average	7	$9 \times 7 = 63$
EIF	3	Average	5	$3 \times 5 = 15$

Unadjusted Function Points (UFP) = $20 + 20 + 12 + 63 + 15 = 130$

3.2.7. Adjust for Complexity

Value Adjustment Factor (VAF) = sum of 14 general system characteristics (0-5 each), typically ranges 0-70. For simplicity, let's assume average complexity VAF = 1.0 (no major adjustments).

Adjusted FP = $UFP \times VAF = 130 \times 1 = 130$ FP

3.3. Effort, Duration, and Cost Estimation

3.3.1. Effort

Assuming productivity rate = 10 FP per person-month

- Effort = $130 / 10 = 13$ person-months

3.3.2. Duration

Assume team size = 3 developers, 1 person-month = 160 hours:

- Total hours = $13 \times 160 = 2080$ hours
- Duration = $2080 / (3 \times 160) \approx \mathbf{4.33 \text{ months}}$ (~4 months + 1 week)

3.3.3. Cost Estimation

Assuming developer salary = PKR 80,000 / month:

- Effort = 13 person-months
- Total cost = $13 \times 80,000 = \mathbf{PKR 1,040,000}$

3.4. Assumptions and Limitations

Assumptions:

- Average complexity for all components.
- Productivity rate = 10 FP/person-month.
- Team of 3 developers.
- No major unforeseen technical challenges.

Limitations:

- Early-stage estimate, may change with detailed requirements.
- Adjustments for technical complexity not included (VAF = 1).
- Real implementation may vary depending on team skill, interruptions, or scope creep.